

AI Assignment 2 Report

Variational Bayesian Last Layers for Handwritten Digit Classification

CISC3024 Pattern Recognition | October 2026

Summary

This report records an AI-assisted implementation and experiment for the assignment. I asked an AI tool to identify a recent Bayesian classification method, checked the method against its ICLR 2024 paper and author-maintained code, generated the Python implementation, and used the resulting run to prepare this report. The selected method is Variational Bayesian Last Layers (VBLL), which represents uncertainty in the output-layer weights while keeping the learned image features fixed.

On one stratified test split of 450 small handwritten-digit images, the deterministic MLP reached 96.89% accuracy and VBLL reached 96.67%. VBLL had lower 15-bin expected calibration error (0.0198 versus 0.0267), while its negative log likelihood and Brier score were slightly higher. The run shows a small calibration trade-off; it does not establish that VBLL is better across datasets or random splits.

Test accuracy	Test NLL	Test ECE
VBLL 96.67%	VBLL 0.0998	VBLL 0.0198

Held-out test set comparison. Lower NLL and ECE are better; accuracy is better when higher.

Assignment Coverage

Sections 1 to 6 answer the requested points in order: how AI was asked to find the algorithm, the algorithm, the AI-generated implementation, experiment settings and results, lessons learned, and the source-code webpage.

1 How AI Found the Algorithm

I gave the AI tool the assignment constraints: choose a recent classification method for computer vision or pattern recognition, ground it in probability or Bayesian theory, and prefer a peer-reviewed primary paper with reproducible code. The search compared Bayesian last-layer methods and Laplace-based methods, then focused on Variational Bayesian Last Layers because it is a 2024 ICLR classification method and the authors publish a PyTorch implementation.

The AI checked the ICLR proceedings page and the paper's classification theorem before coding. It also inspected the authors' Vector Institute repository and API documentation. This confirmed that the method places a variational Gaussian distribution on the last-layer weights and trains it with a deterministic bound on the classification evidence. This reproduction implements the discriminative classification variant; it does not reproduce all experiments in the paper.

Representative search prompt

Find a recent classification method for computer vision or pattern recognition that is explicitly based on probability or Bayesian theory. Prefer a 2023-or-later peer-reviewed primary source and an author-maintained implementation. Explain why it fits this assignment and what can be reproduced on a small offline dataset.

The prompt log contains the implementation and report-writing prompts as well. Those prompts asked the AI to use a held-out test set only after validation decisions were fixed and to report trade-offs without assuming that a Bayesian method must win every metric.

2 Algorithm Description

Let $\phi(x; \theta)$ be the feature vector produced by the trained neural network, including a constant bias feature. VBLL places a Gaussian variational posterior on the class-specific last-layer weight rows W . In this reproduction, each class row has an independent diagonal covariance, and the prior is a zero-mean Gaussian with identity covariance. The feature extractor stays deterministic.

$$q(W | \eta) = \text{product}_k \text{Normal}(w_k | \mu_k, S_k), \quad p(W) = \text{product}_k \text{Normal}(w_k | 0, I)$$

For a one-hot class label y_n , Theorem 2 gives a deterministic lower bound for discriminative classification. Here μ stacks the class means μ_k , and S_k is the diagonal covariance for class k . The second term adds half the variance of each class logit inside a log-sum-exp. We set output-noise variance to zero and train the variational mean and covariance by maximizing this bound minus the analytic KL divergence from $q(W)$ to its prior.

$$\text{ELBO} = (1/N) \sum_n [y_n^T \mu \phi_n - \log \sum_k \exp(\mu_k^T \phi_n + 0.5 \phi_n^T S_k \phi_n)] - (1/N) \text{KL}(q(W) || p(W))$$

Training does not sample weights in the loss. At prediction time, the classifier draws W from the fitted $q(W)$, applies softmax, and averages the class probabilities. This approximates the Bayesian predictive probability $E_q[\text{softmax}(W \phi)]$.

3 How AI Implemented the Algorithm

The AI generated ``vbl_digits_classification.py``. It uses the paper's post-training option: first fit an ordinary neural feature extractor, then hold those features fixed and optimize the variational last layer. This separates feature learning from posterior fitting and follows the two-stage procedure described in the paper.

- Load the built-in 8 by 8 digits images, scale pixel values from 0 to 16 into 0 to 1, and form stratified train, validation, and test partitions.
- Train a 64-32-10 multilayer perceptron with ReLU hidden units, cross-entropy loss, Adam, and early stopping on validation NLL.
- Freeze the selected network's 32-dimensional hidden features and append a constant bias feature. Initialize the VBLL variational mean from the trained softmax head.
- Represent each class's output weights by a diagonal Gaussian. Optimize its mean and log variance using the deterministic Jensen classification bound and the analytic Gaussian KL penalty divided by the number of training examples.
- Select the VBLL checkpoint using validation predictive NLL. For the final test prediction, sample 500 weight matrices from $q(W)$, average their softmax probabilities, and calculate the same metrics as for the deterministic MLP.

Implementation Details

The implementation uses PyTorch for the neural network and optimization, scikit-learn for the built-in dataset and stratified splits, and NumPy for metric aggregation. The posterior uses one independent variance per class-weight parameter. A variance of 0.1 initializes $q(W)$; the prior variance is 1.0. The validation predictions use 64 fixed-seed samples per checkpoint so the early-stopping comparison uses common Monte Carlo noise. The held-out test prediction uses 500 samples.

The JSON output stores the split, hyperparameters, package versions, validation metrics, and test metrics. The script also generates the metric chart embedded in this report. The source was executed to obtain the reported experiment values; they are not hand-entered estimates.

4 Experiment Settings and Results

The experiment uses scikit-learn's digits dataset: 1,797 grayscale images, each 8 by 8 pixels, with ten labels and pixel intensities from 0 to 16. The data are divided once with stratified sampling and random seed 42. Training uses 1,077 images, validation uses 270, and the untouched test set contains 450 images.

Setting	Value
Input and network	64 inputs; one 32-unit ReLU hidden layer; 10 classes
Training	Adam, learning rate 0.01, batch size 128, maximum 100 epochs, patience 15
Baseline	Same MLP trained with standard cross-entropy; checkpoint chosen by validation NLL
VBLL head	Frozen MLP features; diagonal Gaussian class rows; prior variance 1.0; zero output-noise variance
Prediction	64 Monte Carlo samples for validation; 500 for the final test probabilities
Metrics	Accuracy, NLL, multiclass Brier score, 15-bin ECE, entropy-based error AUROC

Table 1. Experiment settings. Validation selected checkpoints; the test set was evaluated once after those choices were fixed.

Method	Accuracy	NLL	Brier	ECE
Deterministic MLP	96.89%	0.0962	0.0475	0.0267
VBLL	96.67%	0.0998	0.0488	0.0198

Table 2. Test metrics for 450 held-out images. Lower NLL, Brier, and ECE are better; higher accuracy is better.

VBLL reached 96.67% test accuracy, compared with 96.89% for the MLP. Its ECE was lower, while NLL and Brier score were slightly higher. Entropy-based error-detection AUROC was 0.9523 for VBLL and 0.9517 for the MLP. VBLL's mean predictive entropy was 0.145 nats, versus 0.076 for the MLP, so its predictive probabilities were less concentrated.

Results Interpretation

Digits test set comparison (450 images)

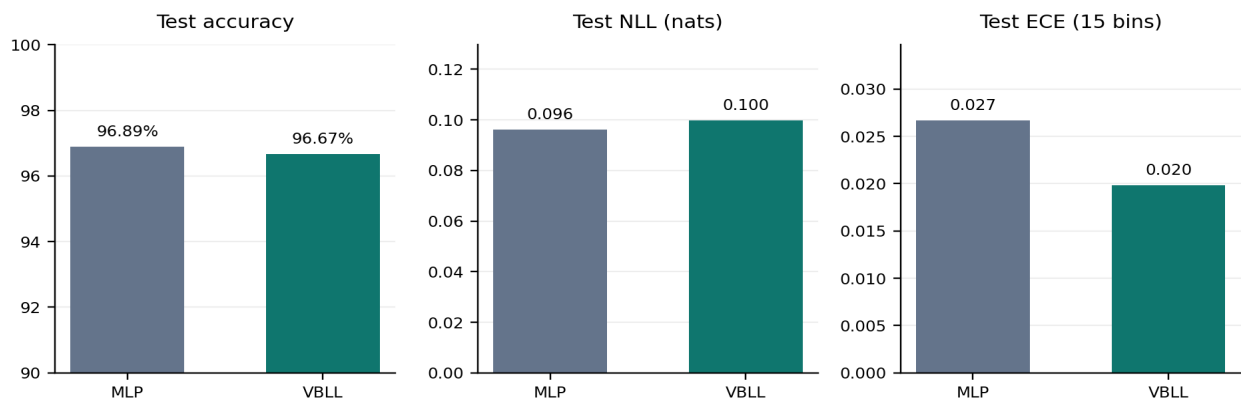


Figure 1. Accuracy, NLL, and ECE on the 450-image test set. Lower NLL and ECE indicate better probabilistic predictions.

The VBLL classifier preserved nearly all of the MLP's accuracy and reduced the measured ECE on this split. Its NLL and Brier score were slightly worse, which shows why accuracy alone is not enough to judge a probabilistic classifier. The result is specific to one small dataset and seed; repeated splits and larger image benchmarks would be needed for a broad performance claim.

5 What I Learned

- A Bayesian last layer can quantify uncertainty in class weights without making every neural-network layer probabilistic.
- The variational objective balances fit to labeled images with a KL penalty that keeps the learned weight distribution near its prior.
- Calibration and classification accuracy measure different properties. VBLL slightly improved ECE here while losing a small amount of accuracy and probability-score performance.
- AI-generated code still needs a reproducible split, validation-only model selection, executable output, and an honest account of limitations.

6 Source Code Webpage

Source code repository: github.com/6657debug/bayesian-pattern-recognition

The repository contains the AI-generated training script, experiment output, metric figure, prompt log, and this report. The code is a small reproduction of the VBLL discriminative classification objective; the authors' full package and paper experiments are linked below.

References

- [1] J. Harrison, J. Willes, and J. Snoek. Variational Bayesian Last Layers. ICLR 2024. [Paper](#)
- [2] Vector Institute. VBLL author-maintained PyTorch implementation. [GitHub](#)
- [3] Scikit-learn. load_digits dataset documentation. [Dataset](#)